

포팅 메뉴얼

목차

목차

1. 개발 환경

Frontend

Backend

Infra

2. 설정 파일 및 환경 변수 설정

3. EC2 연결

4. Docker 설치

5. Docker Compose 설치

6. Jenkins 세팅

7. Backend 컨테이너

8. Nginx 설정

1. 개발 환경

Frontend

- ReactNative CLI
- TypeScript

Backend

- Spring boot 3.4.4
- Spring data jpa 3.4.4
- Spring Security 3.4.4
- JWT 0.12.3
- MySQL 8.0
- QueryDSL 5.0.0

Infra

- EC2
- Docker
- Jenkins

- Nignx

2. 설정 파일 및 환경 변수 설정

- application.yml

```
spring:
  application:
    name: backend

datasource:
  driver-class-name: com.mysql.cj.jdbc.Driver
  url: jdbc:mysql://k12s205.p.ssafy.io/baenana?useSSL=false&useUnicode=true
  username: root
  password: *****

jpa:
  hibernate:
    ddl-auto: none

security:
  oauth2:
    client:
      registration:
        kakao:
          client-id: "*****"
          redirect-uri: "{baseUrl}/login/oauth2/code/kakao"
          client-authentication-method: client_secret_post
          authorization-grant-type: authorization_code
          client-name: kakao
      provider:
        kakao:
          authorization-uri: https://kauth.kakao.com/oauth/authorize
          token-uri: https://kauth.kakao.com/oauth/token
          user-info-uri: https://kapi.kakao.com/v2/user/me
          user-name-attribute: id

jwt:
  secret: *****
```

```
access-expiration: 3600000
refresh-expiration: 604800000
```

```
management:
```

```
  endpoints:
```

```
    web:
```

```
      exposure:
```

```
        include: "prometheus"
```

```
python:
```

```
  service:
```

```
    url: http://3.34.254.33:8000
```

```
rag:
```

```
  service:
```

```
    url: http://3.34.254.33:8000/chat
```

```
    health-check-url: http://3.34.254.33:8000/health
```

3. EC2 연결

```
ssh -i "ssafy-key.pem" ubuntu@j12b105.p.ssafy.io
```

4. Docker 설치

- 우분투 시스템 패키지 업데이트

```
sudo apt-get update
```

- 시스템의 패키지 목록을 최신 상태로 업데이트합니다.

- 필요한 패키지 설치

```
sudo apt-get install apt-transport-https ca-certificates curl gnupg-agent software-properties-common
```

- Docker 설치에 필요한 패키지를 설치합니다. 이 패키지들은 HTTPS를 통해 안전하게 패키지를 다운로드하거나, GPG 키와 저장소를 관리하는 데 사용됩니다.

- **Docker의 공식 GPG 키 추가**

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
```

- Docker 공식 패키지가 위변조되지 않았음을 확인할 수 있도록 GPG 키를 추가합니다.

- **Docker의 공식 apt 저장소 추가**

```
sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"
```

- Docker의 공식 apt 저장소를 우분투 시스템에 추가합니다. 이 저장소는 Docker와 관련된 패키지를 다운로드할 수 있는 출처입니다.

- **시스템 패키지 업데이트**

```
sudo apt-get update
```

- Docker의 저장소가 추가되었으므로, 새로운 패키지 목록을 다시 업데이트합니다.

- **Docker 설치**

```
sudo apt-get install docker-ce docker-ce-cli containerd.io
```

- Docker 엔진 및 관련 패키지를 설치합니다. `docker-ce` 는 Docker Community Edition을 설치하는 패키지입니다.

- **Docker 설치 확인**

- 도커 설치 확인

```
docker --version
```

- 도커 실행상태 확인

```
sudo systemctl status docker
```

- Docker 서비스가 정상적으로 실행 중인지 확인합니다.

- 도커 실행

```
sudo docker run hello-world
```

- Docker가 정상적으로 설치되었는지 확인하기 위해 **Hello World** 컨테이너를 실행합니다. 이 명령어는 Docker가 제대로 작동하는지 검증하는 간단한 방법입니다.

5. Docker Compose 설치

- Docker Compose 설치

```
sudo curl -L https://github.com/docker/compose/releases/download/1.22.0/docker-compose
```

- docker-compose에 권한을 부여

```
sudo chmod +x /usr/local/bin/docker-compose
```

- Version check

```
docker-compose version
```

- Docker Compose 파일 작성

6. Jenkins 세팅

- jenkins 컨테이너 실행

```
docker run -d --name jenkins --privileged -v /var/run/docker.sock:/var/run/docker.sock
```

- 확인

```
docker ps -a
```

- jenkins 컨테이너 접속

```
docker exec -it jenkins bash
```

- 초기 비밀번호 확인

```
cat /var/jenkins_home/secrets/initialAdminPassword
```

- webhook 설정
- 도커 설치

```
apt-get update
apt-get install -y docker.io
```

- 깃랩 연동

publish over SSH 플러그인 설치

```
git clone https://lab.ssafy.com/s12-ai-speech-sub1/S12P21B105
cd S12P21B105
git config --global credential.helper store
```

7. Backend 컨테이너

1. Base 이미지 선택 (빌드용)

FROM openjdk:17-jdk-slim as builder

2. 작업 디렉토리 설정

WORKDIR /app

3. Gradle 빌드 환경 설정 (캐싱 최적화)

COPY gradlew .

COPY gradle gradle

COPY build.gradle settings.gradle ./

RUN chmod +x gradlew

RUN ./gradlew dependencies --no-daemon

4. 프로젝트 코드 복사 및 빌드 실행

COPY src src

RUN ./gradlew build --no-daemon

```
# 5. 실행용 JDK 이미지 설정
FROM openjdk:17-jdk-slim
```

```
# 타임존 설정 추가
ENV TZ=Asia/Seoul
RUN ln -snf /usr/share/zoneinfo/$TZ /etc/localtime && echo $TZ > /etc/timezone
```

```
# 6. 실행 디렉토리 설정
WORKDIR /app
```

```
# 7. 빌드된 JAR 파일 복사
COPY --from=builder /app/build/libs/*.jar app.jar
```

```
# 8. Spring Boot는 기본 8080 포트에서 실행됨
EXPOSE 8080
```

```
# 9. 컨테이너 실행 명령어
ENTRYPOINT ["java", "-jar", "app.jar"]
```

8. Nginx 설정

```
events { worker_connections 1024; }

http {
    include    /etc/nginx/mime.types;
    default_type  application/octet-stream;

    server {
        listen 80;
        server_name k12s205.p.ssafy.io;
        location / {
            return 301 https://$host$request_uri;
        }
    }
}
```

```
server {  
    listen 443 ssl;  
    server_name k12s205.p.ssafy.io;  
  
    ssl_certificate /etc/letsencrypt/live/k12s205.p.ssafy.io/fullchain.pem;  
    ssl_certificate_key /etc/letsencrypt/live/k12s205.p.ssafy.io/privkey.pem;  
  
    location / {  
        proxy_pass http://baenana-backend:8080/;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}  
}
```